

PARTE 1 — Nombres y organización de archivos

Fuente: secciones 2 y 3

Para **cada archivo fuente**, verificar:

- ☒ **Sufijo correcto** — El código fuente usa .java y el bytecode usa .class. (2.1)
- ☒ **Nombres de archivo comunes** — Los makefiles se llaman GNUmakefile y el archivo que resume un directorio se llama README. (2.2)
- ☒ **Una sola clase o interfaz pública por archivo** — Es la primera declarada; las clases/interfases privadas asociadas pueden ir en el mismo archivo. (3.1)
- ☒ **Longitud del archivo** — No supera las 2000 líneas. (3)
- ☒ **Secciones separadas** por líneas en blanco y, opcionalmente, un comentario que identifica cada sección. (3)
- ☐ **Orden del archivo fuente:** (3.1)
 - 1) Comentarios iniciales.
 - 2) Sentencia package y sentencias import.
 - 3) Declaraciones de clases e interfaces.
- ☐ **Comentario inicial** estilo C (`/* ... */`) que incluye: (3.1.1)
 - Nombre de la clase e información de versión.
 - Programador(es) y fecha.
 - Aviso de copyright.
 - Breve descripción del propósito del programa.
- ☒ **Sentencia package** — Es la primera línea que no es comentario; los import van después. (3.1.2)
- ☐ **Orden de las partes de la clase/interfaz:** (3.1.3)
 - 1) Comentario de documentación de la clase (`/** ... */`).
 - 2) Sentencia class o interface.
 - 3) Comentario de implementación de la clase (`/* ... */`), si es necesario.
 - 4) Variables de clase (static): primero public, luego protected, luego private.
 - 5) Variables de instancia: primero public, luego protected, luego private.
 - 6) Constructores.
 - 7) Métodos, agrupados por funcionalidad (no por alcance o accesibilidad).

PARTE 2 — Indentación y longitud de línea

Fuente: sección 4

Revisar que el código **cumpla**:

- ☒ **Unidad de indentación** de 4 espacios. (4)
- ☒ **Tabuladores** configurados cada 8 espacios (no 4), si se utilizan. (4)
- ☐ **Longitud de línea** — Ninguna línea supera los 80 caracteres. (4.1)
- ☒ **Ejemplos para documentación** — Líneas de 70 caracteres como máximo. (4.1)
- ☐ **Corte de líneas largas** según estos principios: (4.2)
 - Cortar después de una coma.
 - Cortar antes de un operador.
 - Preferir cortes de nivel alto (fuera de paréntesis) sobre cortes de nivel bajo.
 - Alinear la nueva línea con el inicio de la expresión del mismo nivel en la línea anterior.
 - Si lo anterior produce código confuso o pegado al margen derecho, indentar 8 espacios.
- ☒ **Declaraciones de métodos largos** — Usan 8 espacios cuando la alineación convencional desplaza demasiado los parámetros. (4.2)
- ☒ **Condiciones de if en varias líneas** — Usan la regla de 8 espacios para que el cuerpo del bloque se distinga con claridad. (4.2)
- ☒ **Operador ternario** con una de las tres formas aceptadas: (4.2)
 - `alpha = (expr) ? beta : gamma;`
 - `? beta` en la misma línea y `: gamma` alineado debajo del `?`
 - `? beta` y `: gamma` cada uno en su propia línea, alineados

PARTE 3 — Comentarios

Fuente: sección 5

Reglas generales para todos los comentarios:

- ☒ **Tipo correcto** — `/* ... */` y `//` para comentarios de implementación; `/** ... */` para comentarios de documentación (javadoc). (5)
- ☒ **Aportan información** que no es evidente en el código; no duplican lo que el código ya expresa claramente. (5)
- ☒ **Sin información que se desactualice** con facilidad ni datos ajenos al programa (cómo se construye el paquete, directorio donde reside). (5)
- ☐ **Decisiones de diseño no obvias** están explicadas cuando corresponde. (5)

- ☒ **Código confuso se reescribe** en lugar de compensarlo con más comentarios. (5)
- ☒ **Sin cajas decorativas** hechas con asteriscos u otros caracteres. (5)
- ☒ **Sin caracteres especiales** como form-feed o backspace. (5)

Comentarios de implementación:

- ☒ **De bloque** — Al inicio de cada archivo y antes de cada método; precedidos de una línea en blanco; con * al inicio de cada línea excepto la primera; indentados al nivel del código que describen. (5.1.1)
- ☒ **De una línea** — Indentados al nivel del código que sigue y precedidos de una línea en blanco; si no caben en una línea, usan formato de bloque. (5.1.2)
- ☒ **Al final de línea (trailing)** — Muy cortos, separados lo suficiente de la sentencia y alineados entre sí; no se comenta cada línea al estilo ensamblador. (5.1.3)
- ☒ **Con //** — No se usan en varias líneas consecutivas para texto; sí se permiten para comentar bloques de código. (5.1.4)

Comentarios de documentación (doc comments):

- ☐ **Presentes en cada clase, interfaz, constructor, método y campo**, uno por API, justo antes de la declaración. (5.2)
- ☒ **Indentación del doc comment** — Clases e interfaces sin indentar (líneas siguientes con 1 espacio); miembros con 4 espacios en la primera línea y 5 en las siguientes. (5.2)
- ☐ **Etiquetas javadoc** (@param, @return, @see) usadas donde aplican. (5.2)
- ☒ **Detalles de implementación fuera del doc comment** — Van en un comentario de bloque o de una línea inmediatamente después de la declaración. (5.2)
- ☒ **Ningún doc comment dentro** del cuerpo de un método o constructor. (5.2)

PARTE 4 — Declaraciones

Fuente: sección 6

Para **cada declaración**, verificar:

- ☒ **Una declaración por línea**, preferentemente acompañada de un comentario. (6.1)
- ☒ **Nunca variables y métodos en la misma línea** (ej. long dbaddr, getDbaddr();). (6.1)
- ☒ **Sin tipos distintos en la misma línea** (ej. int foo, fooarray[];). (6.1)
- ☒ **Declaraciones al inicio de los bloques** { }, no hasta el primer uso. La única excepción es el índice del for. (6.2)
- ☒ **Sin ocultar declaraciones** — No se redeclara en un bloque interno un nombre ya declarado en un nivel superior. (6.2)
- ☒ **Variables locales inicializadas** donde se declaran, salvo que el valor inicial dependa de un cálculo previo. (6.3)

☒ **Formato de clases, interfaces y métodos:** (6.4)

- Sin espacio entre el nombre del método y el paréntesis "(" de sus parámetros.
- La llave "{" al final de la misma línea de la declaración.
- La llave "}" en su propia línea, alineada con la sentencia que abre; en métodos vacíos va inmediatamente después: {}.
- Métodos separados por una línea en blanco.

PARTE 5 — Sentencias

Fuente: sección 7

Para **cada sentencia**, verificar:

- ☒ **Máximo una sentencia por línea** (evitar argv++; argc--;). (7.1)
- ☒ **Sin operador coma** para agrupar sentencias, salvo razón evidente. (7.1)
- ☒ **Sentencias compuestas** — Contenido indentado un nivel más; "{" al final de la línea que las inicia; "}" al inicio de línea, alineada con la sentencia. (7.2)
- ☒ **Llaves siempre** en estructuras de control (if, else, for, while, do), incluso con una sola sentencia. (7.2 / 7.4)
- ☒ **return sin paréntesis**, salvo que aclaren el valor devuelto. (7.3)
- ☒ **if / if-else / else-if** con la forma: (7.4)
 - if (condition) { ... } else if (condition) { ... } else { ... }
- ☒ **for** con la forma for (inicialización; condición; actualización) { ... }; el for vacío termina en punto y coma. (7.5)
- ☒ **Operador coma en el for** con tres variables como máximo; si se requieren más, usar sentencias separadas antes o al final del ciclo. (7.5)
- ☒ **while** con la forma while (condición) { ... }; el while vacío termina en punto y coma. (7.6)
- ☒ **do-while** con la forma do { ... } while (condición);. (7.7)
- ☐ **switch:** (7.8)
 - Cada case que continúa al siguiente (sin break) tiene el comentario /* falls through */ donde iría el break.
 - Siempre incluye un caso default, y este termina con break.
- ☒ **try-catch** con la forma try { ... } catch (ExceptionClass e) { ... }. (7.9)

PARTE 6 — Espacios en blanco

Fuente: sección 8

Revisar que existan **dos líneas en blanco**:

- ☐ Entre secciones de un archivo fuente. (8.1)
- ☒ Entre definiciones de clases e interfaces. (8.1)

Revisar que exista **una línea en blanco**:

- ☒ Entre métodos. (8.1)
- ☐ Entre las variables locales de un método y su primera sentencia. (8.1)
- ☒ Antes de un comentario de bloque o de una línea. (8.1)
- ☒ Entre secciones lógicas dentro de un método. (8.1)

Revisar el uso de **espacios**:

- ☒ **Palabra clave seguida de paréntesis** separada por un espacio (while (true)). (8.2)
- ☒ **Sin espacio entre el nombre de un método y "("** — Así se distinguen las llamadas de las palabras clave. (8.2)
- ☒ **Espacio después de cada coma** en listas de argumentos. (8.2)
- ☒ **Operadores binarios** (excepto el punto) separados de sus operandos por espacios (a = (a + b) / (c * d);). (8.2)
- ☒ **Operadores unarios** (menos unario, ++, --) sin espacio respecto de su operando. (8.2)
- ☒ **Expresiones del for** separadas por espacios (for (expr1; expr2; expr3)). (8.2)
- ☒ **Casts seguidos de un espacio** ((byte) aNum). (8.2)

PARTE 7 — Convenciones de nombres

Fuente: sección 9

Para **cada identificador**, verificar:

- ☒ **Clases** — Sustantivos en mayúsculas y minúsculas, con la primera letra de cada palabra en mayúscula (ImageSprite); simples y descriptivos. (9)
- ☒ **Sin acrónimos ni abreviaturas** en nombres de clase, salvo los más usados que su forma larga (URL, HTML). (9)
- ☒ **Interfaces** — Misma capitalización que las clases (RasterDelegate, Storing). (9)
- ☒ **Métodos** — Verbos con la primera letra en minúscula y la primera letra de cada palabra interna en mayúscula (run(), getBackground()). (9)
- ☐ **Variables** — Primera letra en minúscula y palabras internas con mayúscula (myWidth); cortas, significativas y mnemónicas. (9)
- ☒ **Sin variables de un solo carácter**, excepto temporales: i, j, k, m, n para enteros; c, d, e para caracteres. (9)
- ☒ **Constantes** — Todo en mayúsculas con palabras separadas por guion bajo (MIN_WIDTH, MAX_WIDTH). (9)

PARTE 8 — Prácticas de programación

Fuente: sección 10

Revisar que el código **no contenga**:

- ☒ **Variables de instancia o de clase públicas** sin una buena razón (se aceptan solo si la clase es una estructura de datos sin comportamiento). (10.1)
- ☒ **Acceso a miembros static mediante un objeto** (`anObject.classMethod()`); debe usarse el nombre de la clase. (10.2)
- ☐ **Números mágicos** — Literales numéricos codificados directamente, excepto -1, 0 y 1 como contadores de un for. (10.3)
- ☒ **Asignaciones múltiples** en una sola sentencia (`a.x = b.y = 'c';`). (10.4)
- ☒ **Asignaciones que se confunden con comparaciones**, por ejemplo dentro de la condición de un if. (10.4)
- ☒ **Asignaciones embebidas** para “mejorar el rendimiento” (`d = (a = b + c) + r;`). (10.4)

Revisar que el código **sí aplique**:

- ☒ **Paréntesis** en expresiones con operadores mezclados (`if ((a == b) && (c == d))`). (10.5.1)
- ☒ **Estructura que refleja la intención** — return expresión; en lugar de un if/else que devuelve true/false; ternario cuando corresponda. (10.5.2)
- ☒ **Paréntesis antes del “?”** cuando la expresión contiene un operador binario (`((x >= 0) ? x : -x)`). (10.5.3)
- ☒ **Comentarios especiales** — XXX para algo incorrecto que funciona; FIXME para algo incorrecto y que falla. (10.5.4)

PARTE 9 — Comparación contra el ejemplo de referencia

Fuente: sección 11.1

Comparar **cada archivo** con el ejemplo del estándar:

- ☐ **Estructura general** coincide con el orden: (11.1)
 - Comentario inicial → package → import → doc de la clase → class
 - Comentario de implementación → variables static → variables de instancia
 - Constructores → métodos
- ☐ **Doc comment de la clase** incluye descripción, @version y @author. (11.1)
- ☐ **Cada variable, constructor y método** tiene su propio doc comment. (11.1)

- ☐ **Métodos con parámetros** documentan cada uno con @param. (11.1)

Cómo usar esta checklist

1. Revisa la estructura de cada archivo .java con la **Parte 1** y compárala con la **Parte 9**.
2. Revisa el formato visual (indentación, longitud de línea y espacios) con las **Partes 2 y 6**.
3. Revisa la documentación del código con la **Parte 3**.
4. Recorre declaraciones y sentencias con las **Partes 4 y 5**.
5. Verifica los nombres de clases, interfaces, métodos, variables y constantes con la **Parte 7**.
6. Cierra la revisión con las buenas prácticas de la **Parte 8**.
7. Marca con un clic la casilla de cada punto que tu código **sí cumple**; las casillas que queden vacías son los hallazgos a corregir.

# Parte	# defecto	Línea(s) de código	Archivo	Descripción del defecto
1 (3.1.1)	1	1	FriendController	No hay comentario inicial estilo C (<i>/* ... */</i>) con nombre de la clase, versión, programador(es), fecha, aviso de copyright y breve descripción del propósito.
1 y 9 (3.1.3, 11.1)	2	31	FriendController	La clase no tiene comentario de documentación (<i>/** ... */</i>) con descripción, @version y @author, justo antes de la sentencia class.
2 (4.1)	3	40, 59, 67, 77, 93, 102, 111, 113, 123	FriendController	Líneas de más de 80 caracteres (aprox.: 92, 104, 91, 82, 104, 87, 94, 88 y 93).
2 (4.2)	4	59, 93	FriendController	Los return con ResponseEntity.ok(Pageable.toPageable(...)) no se cortan; deben partirse después de una coma y alinear la continuación.
3 (5.2)	5	32–35	FriendController	Los cuatro campos no tienen doc comment que explique su función.
3 y 9 (5.2, 11.1)	6	37, 62, 96, 105, 117	FriendController	Los cinco métodos públicos no tienen doc comment ni etiquetas @param / @return.
3 (5)	7	56–58, 82–88, 90–92	FriendController	Decisión de diseño sin explicar: sin resultados devuelve 204, pero un campo de búsqueda no soportado (default) devuelve 200 con página vacía.
6 (8.1)	8	2, 27	FriendController	Solo una línea en blanco entre secciones del archivo (package → imports, imports → clase); el estándar pide dos.
6 (8.1)	9	44, 70, 101, 113, 123	FriendController	Falta línea en blanco entre las variables locales y la primera sentencia de cada método.

# Parte	# defecto	Línea(s) de código	Archivo	Descripción del defecto
6 (8.2)	10	77	FriendController	Doble espacio tras “->” en “case PHONE -> pageResult”, inconsistente con la línea 72.
5 (7.8)	11	82–88	FriendController	El default no termina con break (termina con return). Aplicabilidad limitada: el switch usa sintaxis de flecha (Java 14+).
7 (9)	12	33 (uso en 112)	FriendController	La dependencia AddFriendService debe llamarse addFriendService y no addFriend (además coincide con el método addFriend, línea 106).
7 (9)	13	34 (uso en 124)	FriendController	El campo removeFriend (tipo RemoveFriend) coincide con el método removeFriend (línea 118); renombrar, p. ej. removeFriendUseCase.
8 (10.3)	14	40, 67 (y 48, 75, 80)	FriendController	Número mágico: el “10” de defaultValue se repite; extraer una constante (DEFAULT_LIMIT). El 0 de PageRequest.of(0, limit) tampoco es contador de for.